RESEARCH & ANALYSIS DOCUMENT

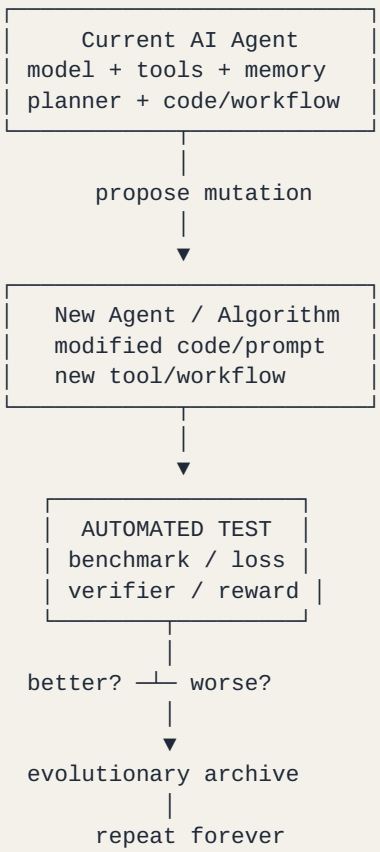
Recursive Self-Improvement in State-of-the-Art AI

Yes. The interesting thing is that “recursive self-improvement” in the current SOTA is not primarily an LLM rewriting its neural weights. The strongest demonstrated systems are closer to evolutionary search over agent programs, workflows, algorithms and training procedures, with LLMs acting as the generator of mutations.

The Sakana AI / UBC Darwin Gödel Machine (DGM) is probably the clearest conceptual example.

The core idea

A modern RSI system looks roughly like:



The crucial insight is:

> *The system doesn't need to understand how to become smarter in the abstract. It needs a mechanism for generating modifications and an objective that tells it whether the modification was better.*

That distinction is extremely important.

---

1. The seven key concepts behind SOTA RSI

1. Self-modification

The system can modify something that determines how it operates.

This could be:

Python code
tools
prompts
planning strategy
memory system
search algorithm
evaluator
training algorithm
architecture
hyperparameters
data-generation procedure
agent orchestration

DGM literally allows the agent to inspect and modify its own codebase.

But this is broader than "AI edits its own weights."

---

2. An external objective / verifier

This is probably the most important practical ingredient.

You need something like:

```
candidate A → 61%
candidate B → 67%
candidate C → 64%
```

so the system knows that B is better.

For coding:

```
run tests
↓
SWE-bench score
↓
accept/reject
```

For mathematics:

```
candidate algorithm
↓
run evaluator
↓
correctness + performance
```

For ML:

```
candidate training algorithm
↓
train model
↓
validation score
```

For science:

```
hypothesis
↓
experiment
↓
measurement
```

This is why verifiable domains are currently much easier for RSI.

AlphaEvolve is a very good example: Gemini proposes programs, automated evaluators execute and score them, and an evolutionary process preferentially uses promising programs to generate further candidates.

---

3. LLM as the mutation engine

This is the subtle part.

The LLM isn't necessarily the thing being evolved.

Instead:

```
existing agent
       ↓
LLM examines it
       ↓
"How could this be improved?"
       ↓
writes modification
       ↓
new agent
```

So the foundation model acts somewhat like a programmer/evolutionary mutation operator.

For example:

```
def solve(task):
    read_files()
    edit_files()
    run_tests()
```

LLM mutation:

```
def solve(task):
    read_files()
    generate_three_solutions()
    rank_solutions()
    edit_files()
    run_tests()
    perform_self_review()
```

Then benchmark it.

If it improves performance:

```
keep it
```

Otherwise:

```
discard / retain as exploratory branch
```

DGM demonstrated exactly this kind of evolution, discovering things such as better editing tools, patch validation, multiple-solution generation/ranking and improved history management.

---

4. Evolutionary archive rather than simple hill climbing

This is one of DGM's most important ideas.

A naive RSI system would do:

```
A
↓
A'
↓
A''
↓
A'''
↓
...
```

Always modifying the current best.

That is ordinary hill climbing.

DGM instead maintains:

```
          A
        /  |        B   C   D
      / \     /      E   F   G   H
         |
         I
```

The archive contains many different agents.

A new candidate can be generated from:

the current best
an unusual branch
a historically successful branch
even a temporarily worse agent

Why?

Because a bad intermediate state may contain the stepping stone to a much better future state.

DGM explicitly found that some lower-performing ancestors led to later breakthroughs, which would have been lost by greedy hill climbing.

This is classic evolutionary/open-ended search.

---

5. Open-endedness

This is where DGM differs conceptually from conventional AutoML.

Normal optimisation:

> *Find the best solution to X.*

Open-ended evolution:

> *Keep generating interesting new solutions, without necessarily knowing what the final solution looks like.*

That creates:

```
variation
   ↓
selection
   ↓
diversity
   ↓
new capabilities
   ↓
new search possibilities
   ↓
more variation
   ↓
...
```

The really interesting property is that the search algorithm itself can become better at searching.

That's the recursive part.

---

6. Meta-improvement

This is arguably the most important concept if you're thinking about genuine RSI.

Imagine the system initially has:

```
Agent
 ├── planner
 ├── coding tool
 ├── memory
 └── evaluator
```

It discovers:

> *"My planner is poor."*

It improves planner.

Then:

> *"My search strategy is poor."*

It improves search.

Then:

> *"I need to generate more diverse candidates."*

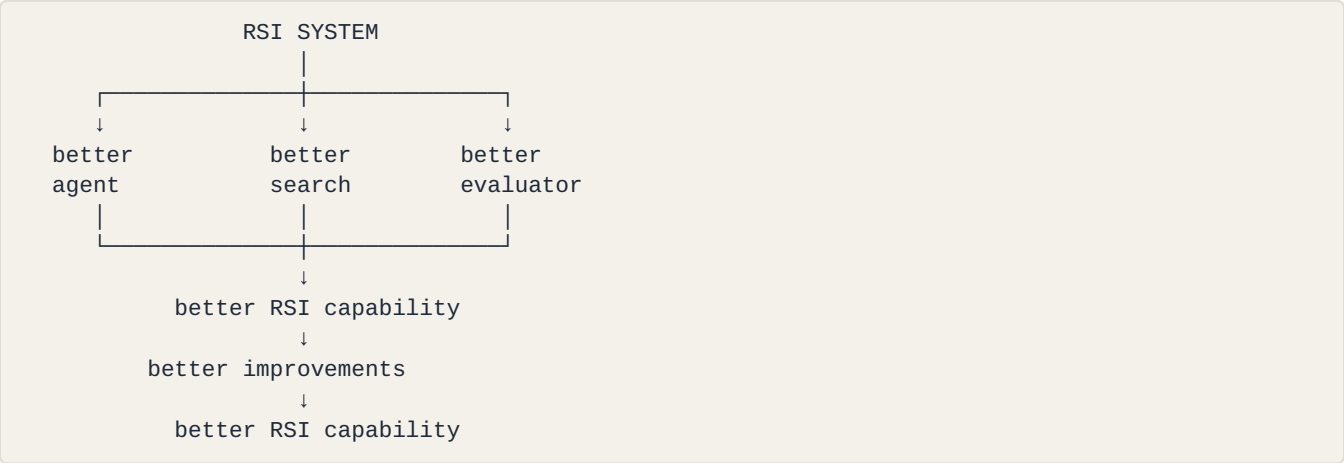
It improves evolutionary search.

Then:

> *"My evaluator isn't sufficiently discriminating."*

It improves evaluation.

Eventually:



That's recursive amplification.

The system isn't merely getting better at the task.

It's getting better at getting better.

---

7. Search + learning + verification

The emerging SOTA architecture is increasingly a hybrid of:

LLM reasoning
evolution
automated experimentation
objective evaluation
memory/archive
parallel search

rather than simply:

> *"Ask a smarter LLM to improve itself."*

DGM vs AlphaEvolve

This distinction is particularly useful.

Dimension	Darwin Gödel Machine	AlphaEvolve
Primary target	AI agent	Algorithms/programs
Self-modifies itself	Yes	Less directly
LLM generates mutations	Yes	Yes
Evolutionary search	Yes	Yes
Archive	Central	Population/database
Automated evaluation	Coding benchmarks	Program evaluators
Open-ended exploration	Central	More optimisation-oriented
Recursive improvement	Explicit	More indirect
Main demonstrated capability	Agent improvement	Algorithm discovery
Key idea	Evolve the agent	Evolve the solution/algorithm

AlphaEvolve has nevertheless gone surprisingly far: Google reports applications to data-centre scheduling, TPU design, matrix multiplication and other algorithms, with the 2026 update reporting continued deployment into infrastructure.

There is another branch: LLM² / AI²

Sakana's earlier LLM-Squared (LLM²) work attacks a different layer.

Instead of:

> *"Improve my agent."*

the system asks:

> *"Can an LLM invent a better algorithm for training LLMs?"*

For example:

```
LLM
 ↓
invent training algorithm
 ↓
train/evaluate
 ↓
better algorithm?
 ↓
retain
 ↓
generate next algorithm
```

Sakana reports that this produced DiscoPOP, an automatically discovered preference-optimisation algorithm.

This is much closer to:

AI designing AI-training algorithms.

And then AI Scientist

Another branch is:

```
AI
 ↓
generate research idea
 ↓
write experiment
 ↓
run experiment
 ↓
analyse result
 ↓
generate next idea
 ↓
repeat
```

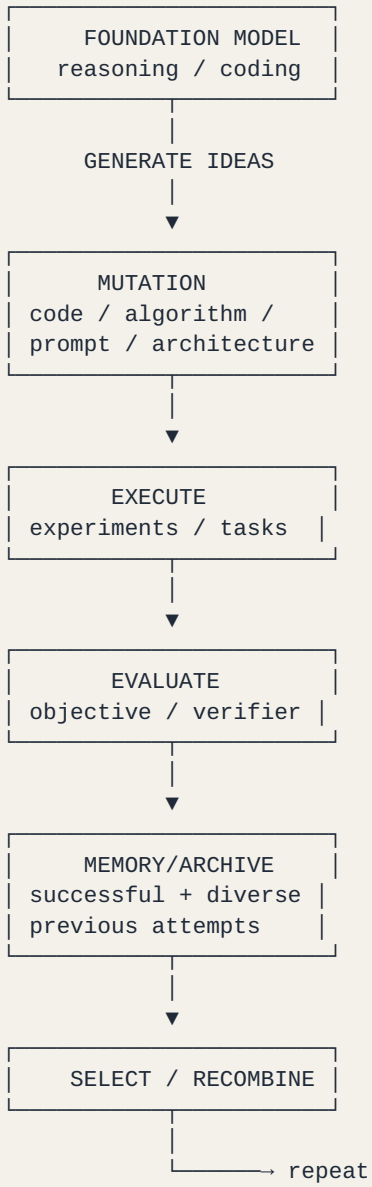
This creates an automated scientific discovery loop.

The important connection is that scientific discovery itself becomes an optimisation/search process.

Sakana now describes these systems—LLM², DGM and AI Scientist—as pieces of a broader RSI trajectory.

The deeper architecture

If you abstract away the names, most of these systems reduce to this:



That is the RSI engine.

What is actually "recursive"?

There are several levels, and people often mix them up.

Level 0 — ordinary inference

```
LLM → answer
```

No self-improvement.

Level 1 — self-reflection

```
LLM → answer → critique → revise
```

Useful, but not really RSI.

Level 2 — workflow optimisation

```
Agent → modify prompt/workflow → benchmark → retain
```

Agent becomes better.

Level 3 — agent-code evolution

```
Agent A
 ↓
Agent B
 ↓
Agent C
```

DGM territory.

Level 4 — search-system improvement

```
agent improves
 ↓
search improves
 ↓
better search discovers better agents
```

Now the improvement process itself improves.

Level 5 — foundation-model improvement

```
AI
 ↓
discovers better training algorithm
 ↓
trains better AI
 ↓
better AI discovers better training algorithm
 ↓
...
```

This is the much more powerful form of RSI that people ultimately care about.

Level 6 — AI research automation

```
AI
 ↓
researches
 ↓
discovers algorithm
 ↓
builds training system
 ↓
trains improved model
 ↓
improved model researches
 ↓
...
```

That starts looking like a genuine AI R&D flywheel.

The biggest bottleneck: the evaluator

This is probably the most important practical insight.

You can generate millions of brilliant ideas.

But if you cannot automatically determine:

> *"Is this actually better?"*

then recursive improvement stalls.

So:

Generation × Evaluation × Compute

is a better mental model than simply "intelligence."

For example:

Domain	Candidate	Evaluator
Coding	new agent	SWE-bench/tests
Algorithms	new algorithm	runtime/correctness
Math	proof/solution	verifier
ML	training method	validation loss
Robotics	controller	simulation
Science	hypothesis	experiment
Agent architecture	new workflow	benchmark
LLM architecture	new architecture	training/eval

This is why coding is currently such a fertile RSI laboratory.

The really interesting frontier

I think the most important transition is:

Current

```
Human designs search space
          ↓
AI searches it
```

AlphaEvolve/DGM move toward:

```
Human designs broad framework
          ↓
AI modifies the search process
          ↓
AI searches
          ↓
AI discovers better search
```

And the eventual target is:

```
AI
  │
  ▼
improve algorithm
  │
  ▼
improve training
  │
  ▼
improve model
  │
  ▼
improve researcher
  │
  ▼
improve RSI machinery
  │
  └─────────────────┐
                    ↓
           better AI
                    ↓
                  ...
```

That's the recursive intelligence explosion mechanism people are ultimately interested in—not simply an LLM editing its own Python file.

And Sakana is now explicitly framing its RSI programme around this progression: agent-native models → AI Scientist → recursive self-improvement → increasingly autonomous AI development.

One particularly important distinction

I would separate three things that are currently all being called RSI:

A. Self-improving agents

> *DGM*

B. Evolutionary algorithm discovery

> *AlphaEvolve / ShinkaEvolve / LLM²*

C. Autonomous AI R&D

> *AI Scientist + automated training/evaluation + model/algorithm evolution*

C is the real endgame.

A system that improves its SWE-bench score from 20% → 50% is impressive, and DGM demonstrated exactly that, but it isn't yet an AI that autonomously redesigns its neural architecture, invents a better training paradigm, trains the successor model and then uses that successor to continue the cycle.

That gap—between recursive improvement of an agent harness and recursive improvement of the underlying intelligence—is arguably the key frontier of RSI research today.

If you're thinking about your own RSI/LLM harness, the practical architecture to copy is therefore much closer to DGM + AlphaEvolve + AI Scientist combined than to a simple "self-reflection agent."